Харківський національний університет радіоелектроніки

Факультет комп’ютерних наук

Кафедра програмної інженерії

ЗВІТ

Звіт до лабораторного заняття №1

з дисципліни

«Штучний інтелект у програмній інженерії»

Виконав:

студент гр. ПЗПІ-23-1

Тітаренко Микита

Перевірили:

доцент кафедри ПІ

Каук Віктор Іванович

Харків 2026

1.1 Тема

Побудова AI-агента: власний цикл Observe, Reason, Act, Evaluate і реалізація на Google ADK.

1.2 Мета

Налаштувати відтворюване середовище розроблення AI-агентів; побудувати на Python (або на іншій мові, але тоді вже самостійно) власного мінімального агента з циклом Observe, Reason, Act, Evaluate без agent framework; відтворити того самого агента на Google ADK з власним інструментом; експериментально порівняти одиничний виклик LLM, власного агента і ADK-агента на п’яти типах запитів; оформити паспорт агента. Всі отримані результати зберегти на GitHub і у відповіді надати вебпосилання на репозиторій.

1.3 Обрана предметна область

Стратегії кешування.

Тема дипломної роботи: вебзастосунок для публікації та перегляду рецептів у форматі соціальної мережі.

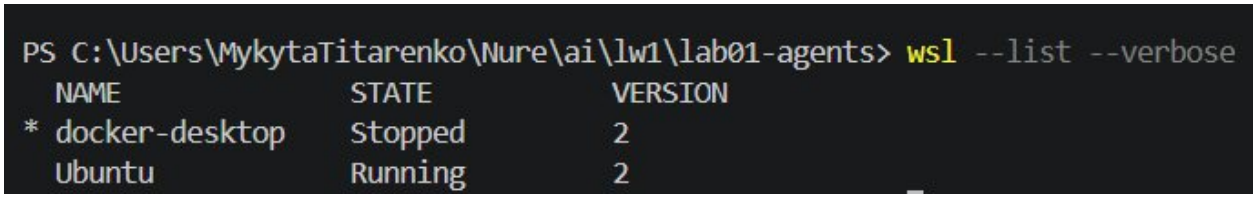
Стратегії кешування безпосередньо забезпечують високу швидкість завантаження стрічки рецептів та профілів авторів у соціальній мережі, мінімізуючи час відгуку системи.

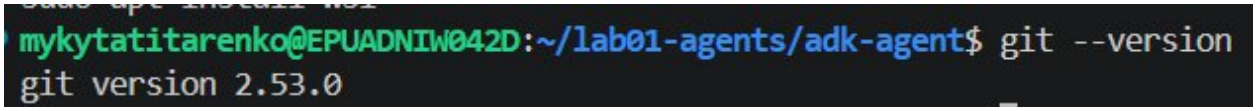
1.4 Конфігурація середовища

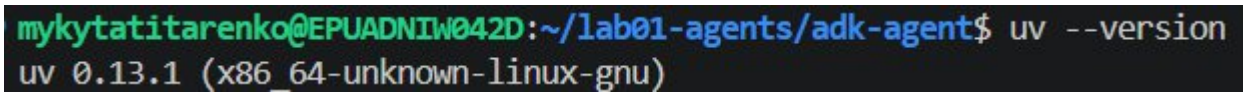
Таблиця 1.1 — Контрольна перевірка середовища

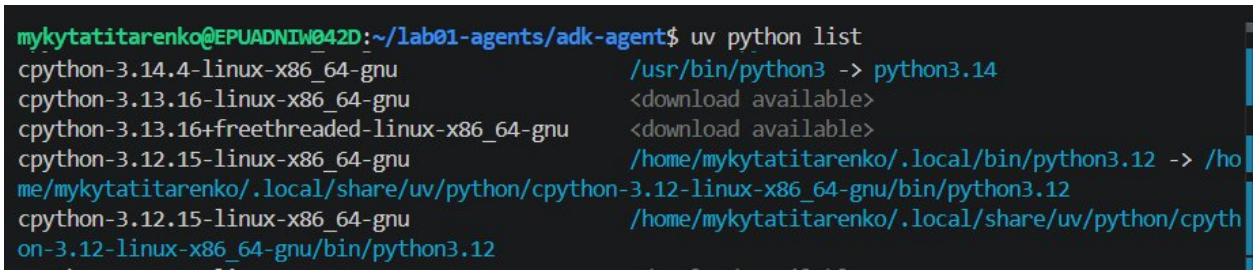
Команда	Очікуваний результат
wsl --list --verbose	Ubuntu, VERSION 2
git --version	git version 2.45.1.windows.1

uv --version	uv 0.13.1
uv python list	cpython-3.12
agents-cli --version	version 1.10.0
git check-ignore -v .env	.gitignore:1:.env .env

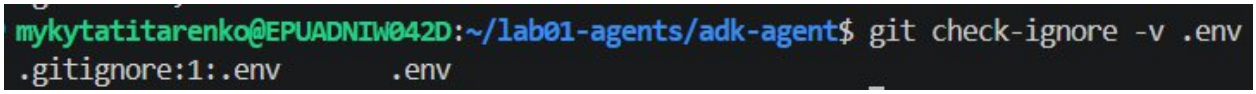


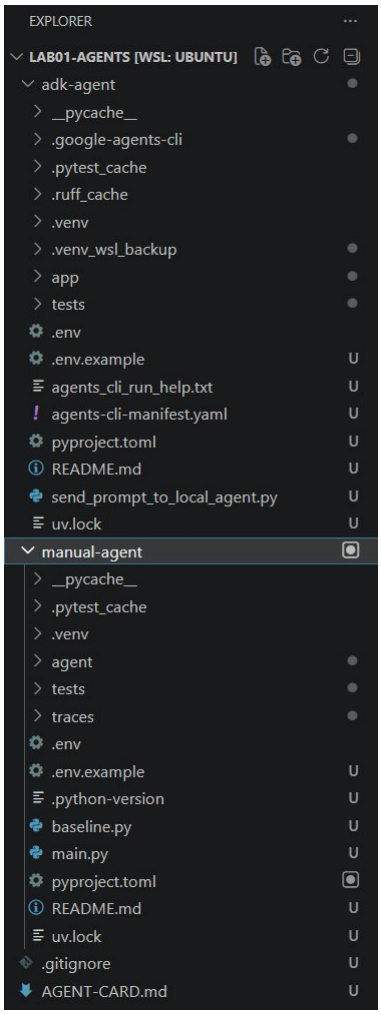












1.5 Архітекутрна схема

USER → GOAL → STATE → OBSERVE → LLM → DECISION → VALIDATOR → DISPATCHER → ENVIRONMENT → OBSERVATION → STATE UPDATE → STOP

1.6 Середовище

Перелік об’єктів:

Cache-Aside, Read-Through, Write-Through, Write-Behind, Write-Around, Refresh-Ahead.

CachingStrategy - клас, який описує одну конкретну стратегію кешування:

- identifier — унікальний код стратегії, наприклад cache_aside
- title — людська назва, наприклад Cache-Aside
- description — короткий опис концепції
- read_path — як відбувається читання
- write_path — як відбувається запис
- strengths — переваги
- tradeoffs — словник недоліків / компромісів
- sources — список джерел, звідки взято факти

Дежерела фактів (зазначені також у вигляді коментарів у коді):

- Microsoft Learn, “Cache-Aside pattern”
- Microsoft Learn, “Caching guidance for application design”
- Microsoft Learn, “No Caching antipattern”
- AWS Prescriptive Guidance, “Database Caching Strategies Using Redis”
- Martin Kleppmann, “Designing Data-Intensive Applications”

Дія lookup_caching_strategy:

```
def _general_method(strategy_name: str) -> dict[str, object] |
```

```
str:
        """Return the full verified record for one caching
strategy."""

        normalized_name = normalize_strategy_name(strategy_name)
        strategy = STRATEGIES.get(normalized_name)
        if strategy is None:
            return NOT_FOUND
        return strategy.to_dict()

    def lookup_caching_strategy(strategy_name: str) -> dict[str,
object] | str:
        """Return the public strategy record without implementation
catalogs."""

        result = _general_method(strategy_name)
        if result == NOT_FOUND:
            return NOT_FOUND

        return {
            "identifier": result["identifier"],
            "title": result["title"],
            "description": result["description"],
            "read_path": result["read_path"],
            "write_path": result["write_path"],
            "strengths": result["strengths"],
            "tradeoffs": result["tradeoffs"],
            "sources": result["sources"],
        }
```

1.7 AgentState, контракт рішення, інструкція компонента рішень

AgentState:

```
@dataclass(frozen=True)
```

```
class Decision:
    action: str
    arguments: dict[str, str]
    decision_summary: str


@dataclass
class AgentState:
    goal: str
    observations: list[str] = field(default_factory=list)
    trace: list[dict[str, Any]] = field(default_factory=list)
    step: int = 0
    llm_calls: int = 0
    finished: bool = False
    final_answer: str | None = None
    stop_reason: str | None = None
```

Контракт рішення:

```
{
  "action": "lookup_caching_strategy",
  "arguments": {
    "strategy_name": "write-through"
  },
```

```
  "decision_summary": "Need the strategy description"
}
```

Інструкція компонента рішень:

```
SYSTEM_INSTRUCTION = """
You are the reasoning component of a manual AI agent for caching strategies.
Return JSON only with keys: action, arguments, decision_summary.
Available actions:
- lookup_caching_strategy with arguments.strategy_name
- get_strategy_tradeoffs with arguments.strategy_name
- finish with arguments.answer when the answer is complete
Rules:
- Use lookup_caching_strategy before answering factual questions about a specific strategy.
- Use get_strategy_tradeoffs when the user asks about the trade-offs, downsides, or operational cost of a strategy.
- If the user asks for comparison, you may call the lookup action multiple times across steps.
- Use only facts from VERIFIED OBSERVATIONS in the final answer.
- If an observation starts with NOT_FOUND, say the information is missing and do not invent it.
- If an observation starts with ERROR, fix your previous decision.
- Do not repeat a lookup that is already present in the
```

```
observations.

- When you finish, write the user-facing answer in Ukrainian in
arguments.answer.

- Keep decision_summary concise and grounded in observations.

""".strip()
```

1.8 Валідатор і диспетчер

Валідатор відповідає за перевірку, чи є рішення агента “коректним” і безпечним для виконання.

- чи дія (action) є допустимою;
- чи arguments — це словник;
- чи для дій lookup_caching_strategy і get_strategy_tradeoffs є обов’язкове поле strategy_name;
- чи для finish є обов’язкове поле answer;
- чи decision_summary не порожній, якщо завершення роботи відбувається.

```
def _require_non_empty_string(arguments: dict[str, str], key: str,
action: str) -> str:

    value = arguments.get(key)

    if not isinstance(value, str) or not value.strip():

        raise DecisionError(f"Action '{action}' requires a non-
empty string argument '{key}'.")

    return value.strip()
```

```
def validate_decision(decision: Decision) -> Decision:
    if decision.action not in ALLOWED_ACTIONS:
        raise DecisionError(
            f"Unknown action '{decision.action}'. Allowed actions: {sorted(ALLOWED_ACTIONS)}."
        )

    if not isinstance(decision.arguments, dict):
         raise DecisionError("Decision arguments must be a JSON object.")

         if decision.action in {"lookup_caching_strategy", "get_strategy_tradeoffs"}:
                    _require_non_empty_string(decision.arguments, "strategy_name", decision.action)

    if decision.action == "finish":
          _require_non_empty_string(decision.arguments, "answer", decision.action)
        if not decision.decision_summary.strip():
             raise DecisionError("Action 'finish' requires a non-empty decision summary.")
```

```
    return decision
```

Диспетчер перетворює “вибір рішення” на конкретну дію.

- перевіряє decision.action;
- викликає відповідну функцію із середовища;
- оновлює стан агенту.

```
def dispatch_action(state: AgentState, decision: Decision) -> str:
    if decision.action == "finish":
        state.final_answer = decision.arguments["answer"]
        state.stop_reason = "finish"
        state.finished = True
        return f"Goal finished: {state.final_answer}"


    if decision.action == "lookup_caching_strategy":
        strategy_name = decision.arguments["strategy_name"]
        result = lookup_caching_strategy(strategy_name)
        if result == NOT_FOUND:
            return f"lookup_caching_strategy('{strategy_name}')
-> {NOT_FOUND}"
        formatted_result = json.dumps(result, ensure_ascii=False,
indent=2)
            return "lookup_caching_strategy result:\n" +
formatted_result
```

```
    if decision.action == "get_strategy_tradeoffs":
        strategy_name = decision.arguments["strategy_name"]
        result = get_strategy_tradeoffs(strategy_name)
        if result == NOT_FOUND:
            return f"get_strategy_tradeoffs('{strategy_name}') -> {NOT_FOUND}"
        formatted_result = json.dumps(result, ensure_ascii=False, indent=2)
                return "get_strategy_tradeoffs result:\n" + formatted_result


      raise DecisionError(f"Unsupported action in dispatch: {decision.action}")
```

1.9 Цикл агента, умови зупинки, обґрунтування MAX_STEPS

Цикл агента:

На кожній ітерації агент:

- перевіряє ліміти зупинки.
- інкрементує step та лічильник LLM-викликів.
- отримує рішення моделі та валідує.
- виконує одну дію;
- логує у trace.

```
def run_agent(
    goal: str,
    *,
    decide_fn: DecisionFunction | None = None,
    max_steps: int = MAX_STEPS,
    max_llm_calls: int = MAX_LLM_CALLS,
    trace_dir: Path | None = None,
) -> AgentState:
    state = AgentState(goal=goal)
    chooser = decide_fn or decide_with_gemini


    while not state.finished:
        if state.step >= max_steps:
            state.finished = True
            state.stop_reason = "max_steps"
            state.final_answer = (
              f"The agent stopped because it reached the step limit
({max_steps}) "
                "without producing a final answer."
            )
            break
```

```
        if state.llm_calls >= max_llm_calls:
            state.finished = True
            state.stop_reason = "max_llm_calls"
            state.final_answer = (
               f"The agent stopped because it reached the LLM call limit ({max_llm_calls}) "
                "without producing a final answer."
            )
            break

        state.step += 1
        decision: Decision | None = None
        started = time.perf_counter()

        try:
            state.llm_calls += 1
                decision = validate_decision(chooser(state.goal, state.observations, state.step, max_steps))
            outcome = dispatch_action(state, decision)
            record = {
                "step": state.step,
                "action": decision.action,
```

```
                "arguments": decision.arguments,
                "decision_summary": decision.decision_summary,
            }
        except (DecisionError, KeyError, TypeError, ValueError,
AttributeError) as error:
            decision = Decision(
                action="rejected",
                arguments={},
                decision_summary=f"Decision error: {error}",
            )
            outcome = f"ERROR: {error}"
            record = {
                "step": state.step,
                "action": "rejected",
                "arguments": None,
                "decision_summary": "",
                "rejected": {
                    "action": decision.action,
                    "arguments": decision.arguments,
                  "decision_summary": decision.decision_summary,
                },
            }
```

```
        record["observation"] = outcome
        record["seconds"] = round(time.perf_counter() - started,
2)
        state.trace.append(record)
        if outcome is not None:
            state.observations.append(outcome)

    save_trace(state, trace_dir=trace_dir)
    return state
```

Умови зупинки:

- state.finished = True;
- state.step >= max_steps;
- state.llm_calls >= max_llm_calls.

MAX_STEPS заданий як 7, адже всього є 6 стратегій кешування і якщо користувач запитає про кожну, то модель зможе отримати інформацію про кожну, а потім витратити ще один крок для сформулювання відповіді. Звісно користувач може запитати про більше стратегій, але тоді агент коректно зупиниться за лімітом кроків і поверне контрольоване повідомлення про досягнення межі, замість нескінченного циклу або частково вигаданої відповіді.

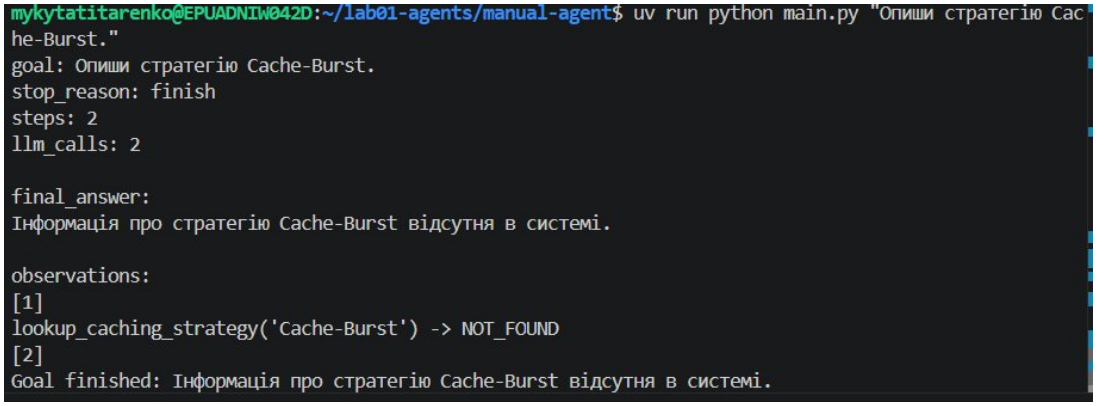
1.10 Багатокрокове трасування

У цьому trace агент виконав три кроки: два lookup_caching_strategy і один finish. На першому кроці він отримав перевірені дані про Cache-Aside, на другому — про Write-Through.На третьому кроці модель сформувала фінальне порівняння на основі зібраних observation без додаткових tool-викликів. Підсумок запуску: stop_reason=finish, steps=3, llm_calls=3.



```
{
  "goal": "Порівняй Cache-Aside і Write-Through.",
  "created_at": "2026-10-20T13:34:26.472015+00:00",
  "stop_reason": "finish",
  "steps": 3,
  "llm_calls": 3,
  "final_answer": "Порівняння стратегій Cache-Aside та Write-Through:\n\n1. **Cache-Aside (лін
  "trace": [
    {
      "step": 1,
      "action": "lookup_caching_strategy",
      "arguments": {
        "strategy_name": "Cache-Aside"
      },
      "decision_summary": "Починаю збір інформації про стратегії кешування для подальшого порі
      "observation": "lookup_caching_strategy result:\n{\n  \"identifier\": \"cache_aside\",\n
      "seconds": 8.71
    },
    {
      "step": 2,
      "action": "lookup_caching_strategy",
      "arguments": {
        "strategy_name": "Write-Through"
      },
      "decision_summary": "I have successfully retrieved information for Cache-Aside. Now I ne
      "observation": "lookup_caching_strategy result:\n{\n  \"identifier\": \"write_through\",
      "seconds": 9.62
    },
    {
      "step": 3,
      "action": "finish",
      "arguments": {
        "answer": "Порівняння стратегій Cache-Aside та Write-Through:\n\n1. **Cache-Aside (лін
      },
      "decision_summary": "Порівняв Cache-Aside та Write-Through на основі отриманих даних, зо
      "observation": "Goal finished: Порівняння стратегій Cache-Aside та Write-Through:\n\n1.
      "seconds": 17.26
    }
```


1.11 Обробка двох відмов



```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run python main.py "Опиши стратегію Cac
he-Burst."
goal: Опиши стратегію Cache-Burst.
stop_reason: finish
steps: 2
llm_calls: 2

final_answer:
Інформація про стратегію Cache-Burst відсутня в системі.

observations:
[1]
lookup_caching_strategy('Cache-Burst') -> NOT_FOUND
[2]
Goal finished: Інформація про стратегію Cache-Burst відсутня в системі.
```

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run python main.py "Порівняй Cache-Asid
e і Write-Through."
goal: Порівняй Cache-Aside і Write-Through.
stop_reason: max_steps
steps: 2
llm_calls: 2

final_answer:
The agent stopped because it reached the step limit (2) without producing a final answer.

observations:
[1]
lookup_caching_strategy result:
{
  "identifier": "cache_aside",
  "title": "Cache-Aside",
  "description": "Cache-Aside is a lazy-loading pattern in which the application checks the cache fi
rst and, on a miss, reads the value from the primary data store and then writes it into the cache. T
his is the core model described by Microsoft Learn in the Cache-Aside pattern and is often used as a
 baseline for read-heavy systems where the cache is filled only when data is actually needed.",
  "read_path": "On a cache miss, the application reads the value from the database or object store a
nd then populates the cache for future requests.",
  "write_path": "Writes go to the primary store first; the application or cache invalidation logic m
ust then decide whether to refresh or evict the cached entry.",
  "strengths": "This pattern is simple to adopt, works well for skewed read-heavy workloads, and avo
ids caching data that is never accessed.",
  "tradeoffs": {
    "staleness_risk": "The cache can serve stale data if the data store changes and the application
does not invalidate or refresh the entry promptly.",
    "consistency_cost": "Cache invalidation and refresh logic move into the application layer, which
 increases the burden of read-after-write consistency.",
    "cache_miss_penalty": "Cold-cache and miss-heavy workloads incur an initial database read before
 the next requests benefit from the cache.",
    "write_complexity": "The application must coordinate write paths, invalidation, and cache warmin
g or eviction decisions."
  },
  "sources": [
    "Microsoft Learn, 'Cache-Aside pattern'",
    "Microsoft Learn, 'Caching guidance for application design'",
    "AWS Prescriptive Guidance, 'Database Caching Strategies Using Redis'"
```

1.12 Результат модульних тестів

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run pytest
================================== test session starts ==================================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0
rootdir: /home/mykytatitarenko/lab01-agents/manual-agent
configfile: pyproject.toml
plugins: anyio-4.15.1
collected 13 items

tests/test_cli_timing.py ..                                                      [ 15%]
tests/test_environment.py ....                                                   [ 46%]
tests/test_loop.py ....                                                          [ 76%]
tests/test_validator.py ...                                                      [100%]

================================== 13 passed in 0.42s ===================================
```

1.13 ADK-агент

Таблиця 1.2 — Призначення файлів ADK-проєкту

Файл	Що це	У Git?
app/agent.py	Визначення агента: модель, інструкція, інструменти, підключенні тулс	Так

app/environment.py	Зберігає верифіковані дані й функції-інструменти для lookup	Так
app/fast_api_app.py	Піднімає HTTP/FastAPI-інтерфейс, через який цього агента можна викликати як сервіс	Так
pyproject.toml	Метадані проєкту і залежності з діапазонами версій	Так
uv.lock	Точні версії всіх залежностей для відтворюваного встановлення	Так
agents-cli-manifest.yaml	Де лежить код агента (agent_directory) і з якими параметрами створено проєкт	Так
tests/	Модульні, інтеграційні тести і набори оцінювання	Так
.env	Ключ API і параметри Google Cloud	Ні
.venv/	Віртуальне середовище, відтворюється з uv.lock	Ні

Інструкція:

```
INSTRUCTION = """

ROLE: Educational AI agent for caching strategies in distributed systems.

PURPOSE: Answer questions about caching strategies only from verified facts returned by the tools.

LANGUAGE: Always answer in Ukrainian unless the user explicitly requests another language. Even if tool results are in English, paraphrase the final answer in natural Ukrainian.

BEHAVIOR RULES:

1. Before answering about a specific strategy, call lookup_caching_strategy for each mentioned strategy.

2. If a tool result is NOT_FOUND, explicitly say that verified data is unavailable and do not invent facts.
```

```
3. Use get_implementations when the user asks about framework or
library implementations for a strategy and language.

4. In strategy comparisons, use only facts returned by
lookup_caching_strategy or get_implementations.

5. Do not call the same tool again with the same argument unless
the user changes the request.

6. Keep explanations concise and technically precise.

7. Write the final user-facing answer in Ukrainian Cyrillic, not
in English transliteration.

LIMITATIONS: Do not invent sources, do not use knowledge outside
the registered tools for concrete strategy claims, do not leave the
final answer in English unless the user explicitly asked for
English, and do not perform actions for which no tool exists.

""".strip()
```

Інструменти: такі ж як і для manual-agent.

Таблиця 1.3 — Обов’язкові перевірки ADK-агента

№	Запит	Очікувана поведінка
1	Що таке кешування?	Відповідь без виклику інструмента
2	Порівняй Cache-Aside і Write-Through.	Два виклики lookup_caching_strategy, відповідь лише з фактів інструмента
3	Опиши стратегію Cache-Burst."	Виклик з name="Cache-Burst", результат NOT_FOUND, агент визнає відсутність даних
4	agents-cli run "Які недоліки у Cache-Aside, Write-Through і Refresh-Ahead?"	Відповідь у терміналі без помилок

Результати lint:

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/adk-agent$ agents-cli lint
  ▸ uv sync --dev --extra lint
Resolved 57 packages in 4ms
Checked 56 packages in 42ms
  ▸ uv run ruff check .
All checks passed!
  ▸ uv run ruff format . --check
11 files already formatted
  ▸ uv run codespell
  ▸ uv run ty check .
All checks passed!
```

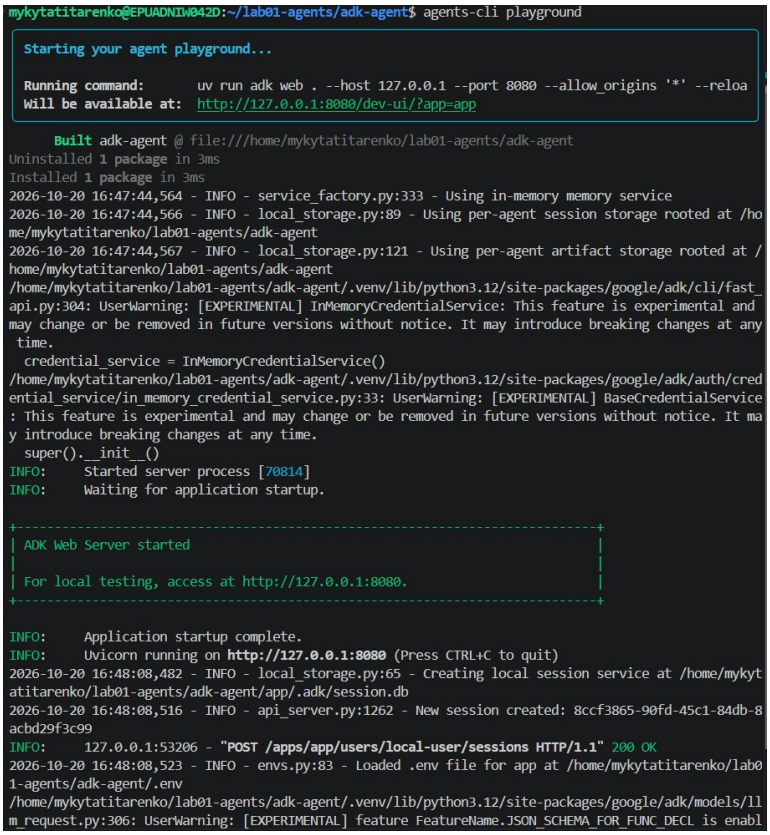
Результати тестів:

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/adk-agent$ uv run pytest tests/unit tests/integration
========================================= test session starts ==========================================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0
rootdir: /home/mykytatitarenko/lab01-agents/adk-agent
configfile: pyproject.toml
plugins: anyio-4.15.1
collected 7 items

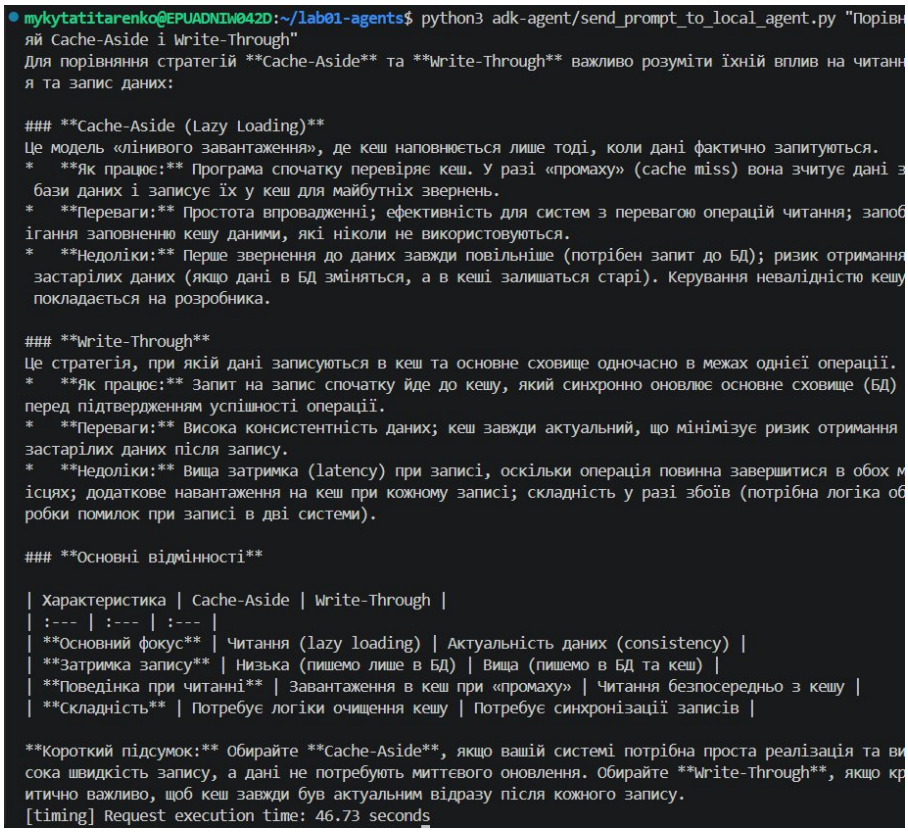
tests/unit/test_dummy.py .                                                              [ 14%]
tests/unit/test_environment.py ....                                                     [ 71%]
tests/unit/test_trace.py .                                                              [ 85%]
tests/integration/test_agent.py .                                                       [100%]

========================================== 7 passed in 1.30s ===========================================
```

Запуск playground:



Виклик send_prompt_to_local_agent.py, який створює сесію та робить запит до серверу:



1.14 Відповідності власного агента і ADK

Таблиця 1.4 — Відповідність елементів власного агента і ADK

Елемент	Частина А (власний код)	Частина Б (ADK)
Мета	Аргумент main.py	Повідомлення користувача в сесії
Стан	AgentState	Сесія: історія подій і словник state
Спостереження	build_observation()	ADK збирає контекст з інструкції, історії подій і результатів інструментів
Контракт рішення	Власний JSON {action, arguments, decision_summary}	Function call у форматі Gemini API; схему побудовано з сигнатури і docstring
Валідація	validate_decision(), білий список	Виконуються лише функції зі списку tools; перевірку значень аргументів пише розробник у самому інструменті
Диспетчер	execute_action()	ADK викликає Python-функцію за назвою

Умова зупинки	Дія finish або MAX_STEPS	Модель відповідає текстом без виклику інструмента; загальний ліміт викликів моделі задає RunConfig.max_llm_calls
Трасування	traces/*.json	Події сесії в Playground
Обсяг коду агента	≈ 410 рядків у 7 модулях пакета agent	≈ 50 рядків у agent.py

1.15 Експерименти

Таблиця 1.5 — Типи запитів

Тип	Призначення	Приклад запиту	Дії / очікувана траєкторія
A	Один об’єкт	"Опиши стратегію Cache-Aside."	lookup_caching_strategy → finish
B	Порівняння двох об’єктів	"Порівняй Cache-Aside і Write-Through."	lookup_caching_strategy × 2 → finish
C	Порівняння trade-offs трьох об’єктів	"Які недоліки у Cache-Aside, Write-Through і Refresh-Ahead?"	lookup_caching_strategy × 3 → finish
D	Об’єкт, якого немає в середовищі	"Опиши стратегію Cache-Burst."	lookup_caching_strategy → NOT_FOUND → finish
E	Загальне питання, цикл зайвий	"Що таке кешування?"	finish одразу або без звернення до середовища

Таблиця 1.6 — Форма запису результатів (фрагмент)

№	Тип	Система	Правильність	Обґрунт.	Галюц.	Виклики LLM	Дії	Час, с	Відтворюваність
1	A	One-shot	2	частково	ні	1	0	27.84	ні

2	A	Власний агент	2	так	ні	2	2	23.20	так
3	A	ADK-агент	2	так	ні	2	1	35.66	так
4	B	One-shot	2	ні	так	1	0	21.10	ні
5	B	Власний агент	2	так	ні	3	3	35.61	так
6	B	ADK-агент	2	так	ні	2	2	9.63	так
7	C	One-shot	2	ні	так	1	0	12.58	ні
8	C	Власний агент	2	так	ні	4	4	17.33	так
9	C	ADK-агент	2	так	ні	2	3	5.76	так
10	D	One-shot	2	так	ні	1	0	23.59	ні
11	D	Власний агент	2	так	ні	2	1	29.61	так
12	D	ADK-агент	2	так	ні	2	1	35.10	так
13	E	One-shot	2	так	ні	1	0	2.76	ні
14	E	Власний агент	2	так	ні	2	1	5.82	так
15	E	ADK-агент	2	так	ні	1	0	3.32	так

1.16 Аналіз експериментів

За результатами можна сказати, що при використанні one-shot модель вигадує інформацію, що можна побачити при додаванні у системний проптм прохання повертати джерела. Наприклад, при промпті "Які недоліки у Cache-Aside, Write-Through і Refresh-Ahead?", модель вказує, що брала джерела з

книзі Мартіна Клеппмана "Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems", але у книзі Мартіна Клеппмана цих термінів та специфічної класифікації немає. Сама книга зосереджена на розподілених системах, реплікації, транзакціях та консистентності даних, терміни Cache-Aside, Write-Through та Refresh-Ahead є стандартними патернами проєктування кешування, які частіше зустрічаються в технічній документації хмарних провайдерів або посібниках з системного дизайну.

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run python main.py "Які недоліки у Cach
e-Aside, Write-Through і Refresh-Ahead?" --mode oneshot
Ось огляд недоліків популярних стратегій кешування:

### 1. Cache-Aside (Lazy Loading)
Це стратегія, при якій програма сама звертається до кешу, а якщо дані відсутні — до бази даних (БД),
 після чого оновлює кеш.

*   **«Прогрів» кешу (Cache Miss):** При першому запиті до нового ключа завжди виникає затримка, оск
ільки дані потрібно спочатку отримати з БД.
*   **Застарілі дані:** Якщо дані в БД змінюються, а кеш не оновлюється або не видаляється (invalida
tion), користувачі отримуватимуть застарілу інформацію.
*   **Складність видалення:** Якщо система розподілена, важко синхронізувати видалення даних з кешу
після оновлення БД, що може призвести до станів «race condition».

### 2. Write-Through
При цій стратегії запис даних спочатку йде в кеш, а потім синхронно в базу даних.

*   **Висока затримка запису:** Кожен запит на запис чекає успішного оновлення обох сховищ (кешу та
БД), що подовжує час виконання транзакції.
*   **Запис "сміття":** Якщо багато даних записуються в систему, але рідко зчитуються, кеш заповнюєт
ься непотрібною інформацією, яка витісняє важливі дані.
*   **Складність підтримки:** Потрібна гарантія атомарності — якщо кеш оновився, а БД ні (або навпак
и), дані стануть неузгодженими.

### 3. Refresh-Ahead
Система автоматично оновлює дані в кеші, передбачаючи майбутні запити, або оновлює їх безпосередньо
перед закінченням терміну життя (TTL).

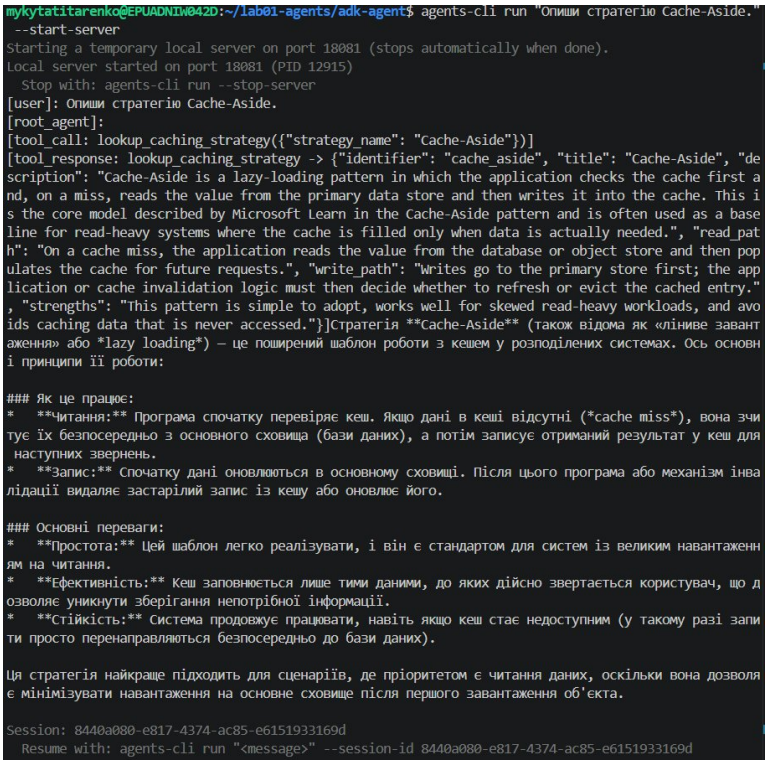
*   **Складність прогнозування:** Якщо алгоритм передбачення помиляється, ресурси витрачаються дарем
но на оновлення даних, які не будуть використані.
*   **Надмірне навантаження на БД:** Якщо кеш занадто великий або часто оновлюється без реальної пот
реби, це створює постійний потік фонових запитів до бази даних.
*   **Ризик отримання неактуальних даних:** Якщо дані в БД оновилися, а «очікуваний» час оновлення к
ешу ще не настав, система може видавати застарілу інформацію протягом певного часу.

---
**Джерело інформації:**
Інформація базується на архітектурних принципах розподілених систем та паттернах проектування високо
навантажених систем.
**Конкретне джерело:** *Kleppmann, M. (2017). "Designing Data-Intensive Applications: The Big Ideas
Behind Reliable, Scalable, and Maintainable Systems" (O'Reilly Media).*
[timing] Request execution time: 2.76 seconds
```

Цікаве спостереження було знайдено у галюцинаціях при one-shot: коли дається одне завдання (A, D, E), то береться джерело в якому дійсно було про це. Коли в запиті декілька стратегій (B, C), то береться джерело в якому не має

про ці кешування. Тому one-shot достатній для невеликих запитів, які вимагають один об’єкт.

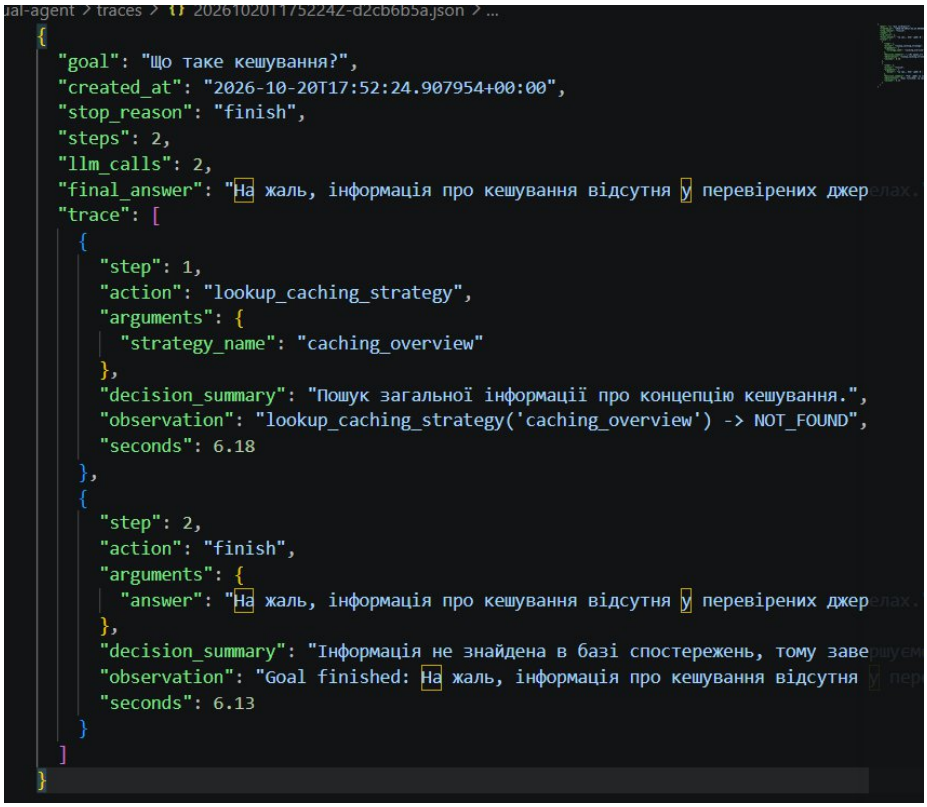
При використанні власного агента чи ADK-агента, то дані беруться тільки з джерел, які були отримані через виклик тулів.



Показники часу чутливі до навантаження моделі та квот провайдера, тому іноді навіть при меншій кількості звернень до LLM час запиту був довшим, тому доцільніше оцінювати кількість LLM запитів. One-shot має найменшу кількість запитів, а у власному агенті найбільша, адже він робить після кожного отримання результату тулу, на відміну від ADK, де викликається відразу декілька.

Також, цікава різниця між manual-agent та ADK-agent для сценрацію E. Мій агент викликав lookup, а ADK відразу дав відповідь, що показує, що модель у ADK зазвичай краще відкалібрована на “коли не викликати тул”,

якщо запит загальний і можна відповісти без отримання інформації з бази.




```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/adk-agent$ agents-cli run "Що таке кешу
вання?" --start-server --verbose
Starting a temporary local server on port 18081 (stops automatically when done).
Local server started on port 18081 (PID 85095)
  Stop with: agents-cli run --stop-server
[user]: Що таке кешування?
[root_agent]: Кешування — це технологія, що дозволяє зберігати копії даних у швидко
му проміжному сховищі (кеші) для того, щоб наступні запити до цих даних виконувалис
я швидше.

Основна мета кешування — зменшити час очікування (latency) та знизити навантаження
на основне джерело даних (наприклад, базу даних або віддалений API), оскільки отрим
ання інформації з кешу відбувається значно швидше, ніж з первинного джерела.

Якщо вас цікавлять деталі щодо конкретних стратегій кешування (наприклад, *Write-Th
rough*, *Write-Back*, *Cache-Aside* тощо) або їх реалізації у певних мовах програму
вання, будь ласка, запитайте, і я надам технічну інформацію з перевірених джерел.
```

1.17 Відповідь на аналітичне питання: «Що саме перетворює програму,

яка викликає LLM API, на AI-агента?».

Програму, що просто викликає LLM API, робить аі-агентом не сам факт генерації тексту, а наявність керованої поведінки в межах задачі. Відмінність починається з мети: агент має чітко визначений результат, наприклад зібрати факти, порівняти стратегії або завершити задачу з перевірюваною відповіддю. Далі потрібен стан, тобто внутрішня пам’ять про вже виконані кроки, отримані дані, ліміти та помилки.

Агент взаємодіє із середовищем через спостереження і дії. Спостереженням є те, що він отримує назовні: результати інструментів, сигнали помилок, проміжні відповіді. Діями є конкретні кроки: виклик інструмента, уточнення, завершення відповіді. Щоб обирати дії послідовно, агенту потрібна політика — правило прийняття рішень на основі поточного стану та нових спостережень. У простому випадку політика описана інструкцією до моделі, у складнішому доповнюється валідацією та обмеженнями.

Усе це працює в циклі агента: спостерігати, оновити стан, обрати дію, виконати, знову спостерігати. Саме цикл перетворює разовий виклик моделі на цілеспрямовану автономну поведінку. Обов’язковою є й умова зупинки: мету досягнуто, вичерпано кроки, перевищено бюджет викликів або виявлено некоректний стан. Фреймворк на кшталт adk або власний цикл реалізує ці елементи технічно: керує станом, інструментами, трасуванням і завершенням. Саме сукупність цих компонентів і формує справжню агентність.

1.18 Висновки

У підсумку створено два варіанти одного агента для задач про стратегії

кешування: manual-agent із власним циклом керування та adk-agent на базі ADK. В обох варіантах узгоджено доменну модель даних, інструменти для пошуку стратегії та отримання імплементацій за мовою, обробку невідомих стратегій, а також тести й трасування виконання.

Локально працюють обидва: запуск агента, тулінг, тести, лінтинг, логування кроків і трас. У manual-agent локально виконується майже вся оркестрація: цикл кроків, валідація дій, ліміти, диспетчеризація інструментів і фіналізація відповіді. Сам інтелектуальний шар відповіді залежить від хмарної LLM-інфраструктури провайдера.

Власний цикл потрібен, коли критично важливі контроль і передбачуваність: жорсткі правила безпеки, детермінований порядок дій, прозорий аудит кожного кроку, кастомні stop-умови та специфічна бізнес-логіка, яку важко виразити декларативно. Це хороший вибір для навчальних експериментів, регульованих доменів і сценаріїв, де потрібна максимально точна керованість поведінки агента.

Фреймворк ADK я б обирав тоді, коли пріоритетом є швидкість розробки, інтеграції та експлуатація: стандартний tool-calling, зручний рантайм, менше власного бойлерплейту, простіше масштабування і підтримка. Для продуктового циклу з частими змінами вимог ADK зазвичай дає кращий баланс між якістю відповіді, UX і вартістю підтримки.



1.19 Просунуті завдання

1 Написати тести циклу агента без звернень до API з підміною функції рішень сценарієм

```
def test_agent_finishes_when_decision_requests_finish(tmp_path: Path) -> None:

    def scripted_decision(_goal: str, _observations: list[str], _step: int, _max_steps: int) -> Decision:

        return Decision(

            action="finish",

            arguments={"answer": "Done"},

            decision_summary="Enough verified observations are
```

```
available.",
        )

         state = run_agent("Коротко поясни Cache-Aside",
decide_fn=scripted_decision, trace_dir=tmp_path)

    assert state.stop_reason == "finish"
    assert state.final_answer == "Done"
    assert state.step == 1

def test_agent_stops_after_max_steps(tmp_path: Path) -> None:
     def scripted_decision(_goal: str, _observations: list[str],
_step: int, _max_steps: int) -> Decision:
        return Decision(
            action="lookup_caching_strategy",
            arguments={"strategy_name": "cache_aside"},
                 decision_summary="Need verified facts before
finishing.",
        )

           state = run_agent("Не завершуй відповідь",
decide_fn=scripted_decision, max_steps=2, trace_dir=tmp_path)
```

```
    assert state.stop_reason == "max_steps"

    assert state.step == 2

    assert "step limit" in state.final_answer


def  test_unknown_action_becomes_observation(tmp_path:  Path)  ->
None:

    decisions = iter(

        [

                        Decision(action="unknown",   arguments={},
decision_summary="bad"),

            Decision(

                action="finish",

              arguments={"answer": "Recovered after the error."},

                decision_summary="The error is already visible in
the observations.",

            ),

        ]

    )


     def scripted_decision(_goal: str, _observations: list[str],
_step: int, _max_steps: int) -> Decision:

        return next(decisions)
```

```
            state = run_agent("Перевір валідатор",
decide_fn=scripted_decision, trace_dir=tmp_path)


    assert state.stop_reason == "finish"

    assert "Unknown action" in state.observations[0]

    assert state.final_answer == "Recovered after the error."


def test_stops_at_max_llm_calls(tmp_path: Path) -> None:

     def scripted_decision(_goal: str, _observations: list[str],
_step: int, _max_steps: int) -> Decision:

        return Decision(

            action="lookup_caching_strategy",

            arguments={"strategy_name": "cache_aside"},

            decision_summary="Need another verified fact.",

        )


    state = run_agent(

        "Keep collecting facts",

        decide_fn=scripted_decision,

        max_steps=5,

        max_llm_calls=2,

        trace_dir=tmp_path,
```

```
        )


        assert state.stop_reason == "max_llm_calls"

        assert state.llm_calls == 2

        assert "LLM call limit" in state.final_answer
```

2 Друга дія:

Для додаткової дії було додані поля implementations та implementation_sources, які відповідають за бібліотеки/фреймворки у певних мовах, які реалізовують відповідний тип кешування. Сама функція get_implementations приймає тип кешування та мову і повертає implementations.

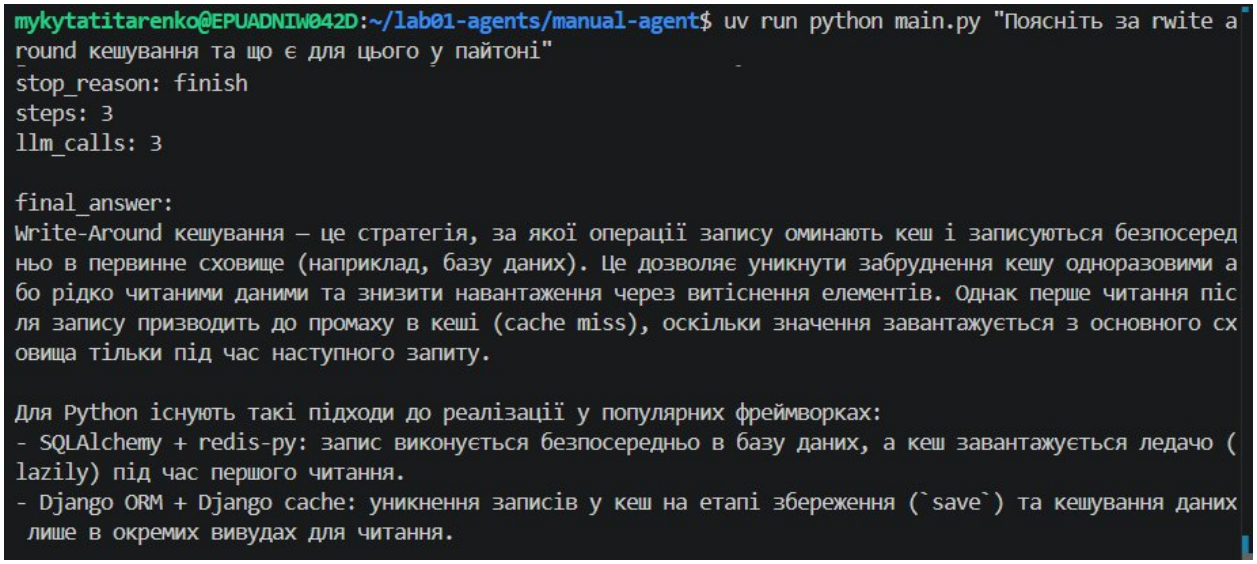
```
    def get_implementations(strategy_name: str, language: str) ->
dict[str, object] | str:
        """Return language-specific implementations and their sources
for a strategy."""

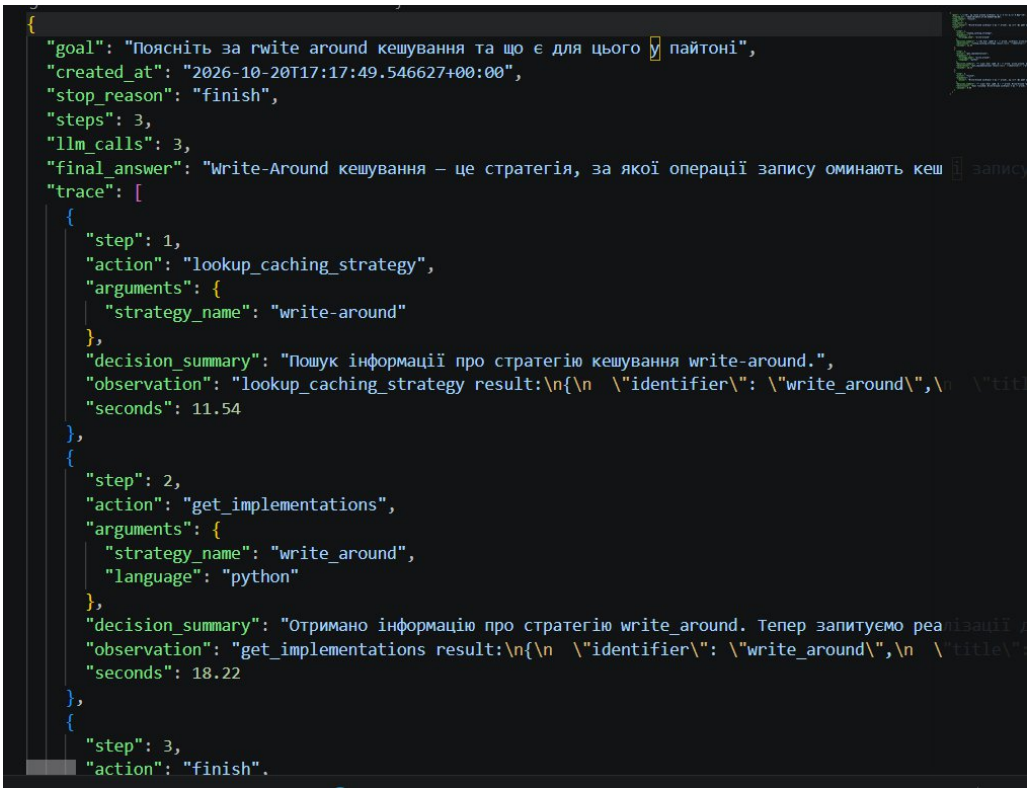
        result = _general_method(strategy_name)
        if result == NOT_FOUND:
            return NOT_FOUND

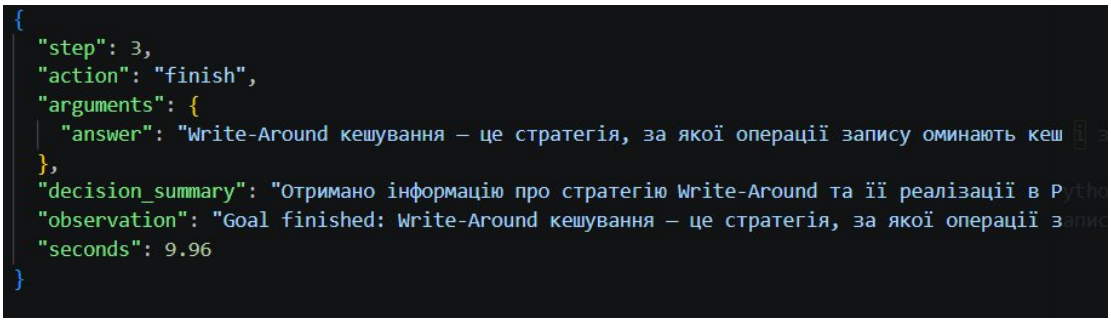
        normalized_language = normalize_language_name(language)
        strategy_implementations = result["implementations"]
        if normalized_language not in strategy_implementations:
            return NOT_FOUND

        return {
            "identifier": result["identifier"],
            "title": result["title"],
```

```
            "language": normalized_language,
            "implementations":
strategy_implementations[normalized_language],
            "implementation_sources":
result["implementation_sources"][normalized_language],
            "sources": result["sources"],
        }
```







3 Було додано MAX_LLM_CALLS, адже це відрізняється від MAX_STEPS чи MAX_ACTIONS:

- MAX_STEPS обмежує загальну кількість ітерацій циклу агента (спостереження, міркування, дія, оновлення стану), щоб він не зациклювався;
- MAX_ACTIONS обмежує саме кількість зовнішніх дій або tool-викликів, тобто контролює операційний вплив на середовище та

інтеграції;

- MAX_LLM_CALLS обмежує число звернень до моделі, що напряму контролює вартість, latency і витрату квоти провайдера.

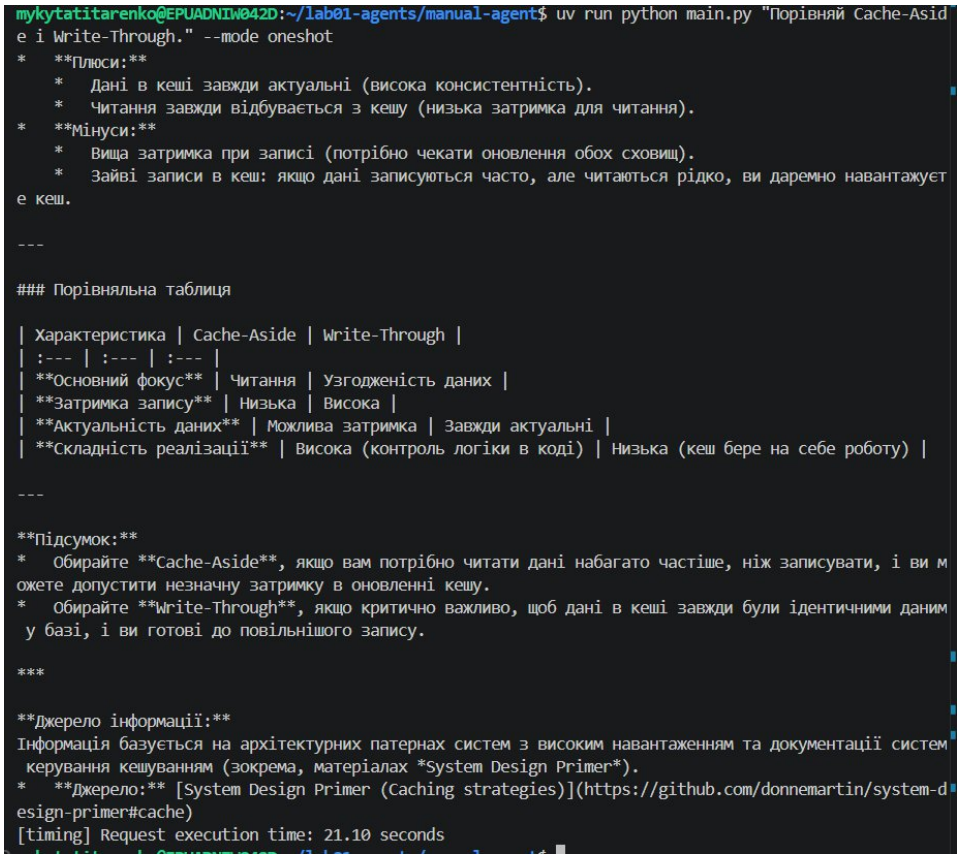
Ці ліміти не взаємозамінні, бо агент може зробити мало кроків, але багато дорогих дій, або багато LLM-викликів без дій.

Тому промислові агенти використовують усі три одночасно, щоб керувати надійністю, безпекою, бюджетом і передбачуваністю з чіткою причиною зупинки.

Щодо MAX_ACTIONS, у поточній реалізації його немає, адже в одному step тільки один tool-виклик.

4 Кожен запит експерименту був проведений тричі.

При використанні one-shot запит був недетермінованим – кожного разу дані бралися з різних джерел. Наприклад, при першому запиті "Порівняй Cache-Aside і Write-Through." дані бралися з System Design Primer, при другому з System Design Interview -An Insider's Guide (Alex Xu), при третьому з матеріалів Microsoft Azure Architecture Guide: Cache-Aside pattern та Write-Through pattern).


mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run python main.py "Порівняй Cache-Aside i Write-Through." --mode oneshot
*   **Плюси:**
    *   Дані в кеші завжди актуальні (висока консистентність).
    *   Читання завжди відбувається з кешу (низька затримка для читання).
*   **Мінуси:**
    *   Вища затримка при записі (потрібно чекати оновлення обох сховищ).
    *   Зайві записи в кеш: якщо дані записуються часто, але читаються рідко, ви даремно навантажуєте кеш.

Порівняльна таблиця
Характеристика	Cache-Aside	Write-Through
Основний фокус	Читання	Узгодженість даних
Затримка запису	Низька	Висока
Актуальність даних	Можлива затримка	Завжди актуальні
Складність реалізації	Висока (контроль логіки в коді)	Низька (кеш бере на себе роботу)

Підсумок:
*   Обирайте **Cache-Aside**, якщо вам потрібно читати дані набагато частіше, ніж записувати, і ви можете допустити незначну затримку в оновленні кешу.
*   Обирайте **Write-Through**, якщо критично важливо, щоб дані в кеші завжди були ідентичними даним у базі, і ви готові до повільнішого запису.
***
Джерело інформації:
Інформація базується на архітектурних патернах систем з високим навантаженням та документації систем керування кешуванням (зокрема, матеріалах *System Design Primer*).
*   **Джерело:** [System Design Primer (Caching strategies)](https://github.com/donnemartin/system-design-primer#cache)
[timing] Request execution time: 21.10 seconds

```
mykytatitarenko@EPUADNIW042D:~/lab01-agents/manual-agent$ uv run python main.py "Порівняй Cache-Asid
e і Write-Through." --mode oneshot

### 2. Write-Through
Ця стратегія забезпечує вищий рівень узгодженості даних, оскільки кеш завжди синхронізований з основ
ною базою.

*   **Як це працює:** Коли програма оновлює дані, вона записує їх одночасно і в кеш, і в базу даних.
 Читання завжди йде через кеш.
*   **Плюси:**
    *   **Узгодженість:** Дані в кеші ніколи не є "старими".
    *   **Швидкість читання:** Дані завжди готові в кеші, тому затримок при читанні немає.
*   **Мінуси:**
    *   **Затримка при записі:** Будь-яка операція запису повільніша, бо потрібно оновити два джерел
а.
    *   **Надлишковість:** У кеш записуються всі дані, навіть ті, що ніколи не будуть прочитані (зас
мічення пам'яті).

---

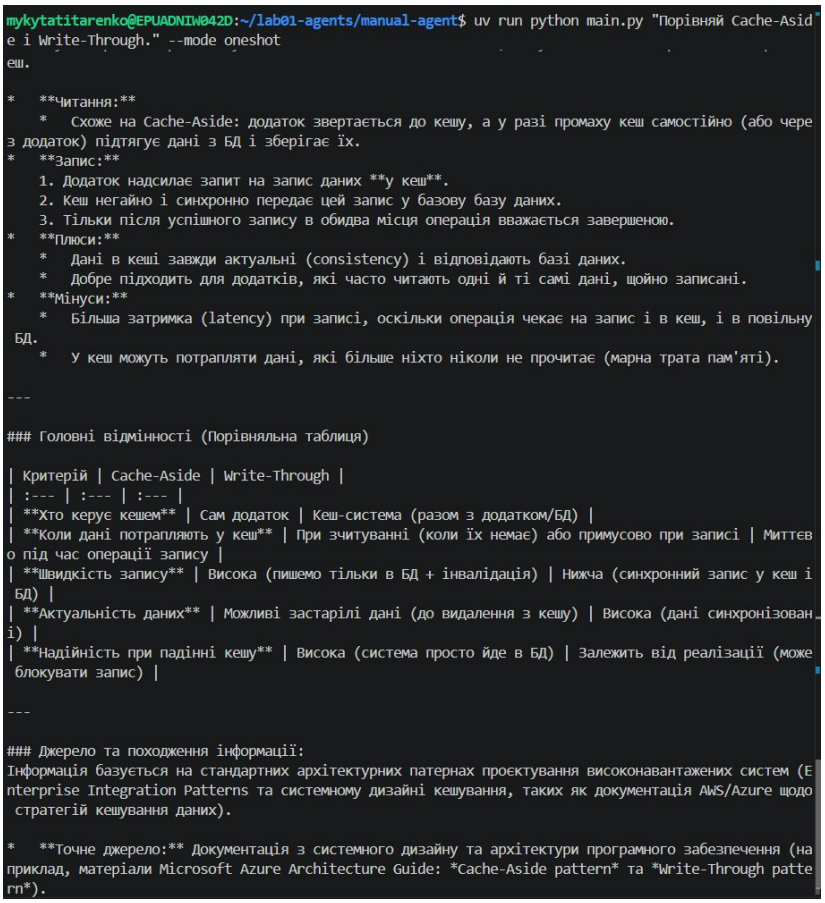
### Порівняльна таблиця

| Характеристика | Cache-Aside | Write-Through |
| :--- | :--- | :--- |
| **Складність реалізації** | Низька | Середня |
| **Узгодженість даних** | Середня (можливі старі дані) | Висока |
| **Швидкість читання** | Повільніша (при першому запиті) | Швидка |
| **Швидкість запису** | Швидка | Повільніша |
| **Надійність (failover)** | Висока | Низька (залежність від кешу) |

---

### Підсумок
*   Використовуйте **Cache-Aside**, якщо вам потрібна висока відмовостійкість і система переважно пр
ацює на читання.
*   Використовуйте **Write-Through**, якщо дані критично важливі та повинні завжди збігатися в кеші
й базі, а затримки при записі не є критичними.

---
**Джерело інформації:**
Дані засновані на архітектурних паттернах проєктування систем та документації з кешування розподілен
их систем.
*   **Джерело:** *System Design Interview – An Insider's Guide* (Alex Xu).
[timing] Request execution time: 25.16 seconds
```



5 Підключення локальної моделі було зроблено через OpenAI-сумісний інтерфейс Ollama, але всередині ADK-агента. Спочатку в агенті додали перемикач backend, щоб можна було запускати або Gemini, або локальний endpoint. Далі для локального режиму виставили змінні оточення: backend = ollama, base URL = локальний сервер Ollama на 127.0.0.1:11434/v1, модель = llama3.2:1b. Поле API key залишили як технічну заглушку (рядок на кшталт ollama), бо для локального Ollama справжній ключ не потрібен, але OpenAI-сумісний клієнт очікує непорожнє значення. Після цього модель завантажили через ollama pull, і запити агента почали йти в локальну Llama.

```
curl -fsSL https://ollama.com/install.sh | sh

ollama serve

ollama pull llama3.2:1b

def _build_model() -> Any:

        if MODEL_BACKEND == "gemini":

            return Gemini(

                model=GEMINI_MODEL,

                retry_options=types.HttpRetryOptions(attempts=3),

            )


           if MODEL_BACKEND in {"openai", "openai_compatible",
"ollama"}:

           # Ollama exposes an OpenAI-compatible endpoint at /v1.

            from google.adk.integrations.openai import OpenAILlm


            return OpenAILlm(

                model=OPENAI_MODEL,

                base_url=OPENAI_BASE_URL,

                api_key=OPENAI_API_KEY,

            )


        raise ValueError(
```

```
                "Unsupported MODEL_BACKEND. Use one of: gemini,
openai_compatible, ollama"

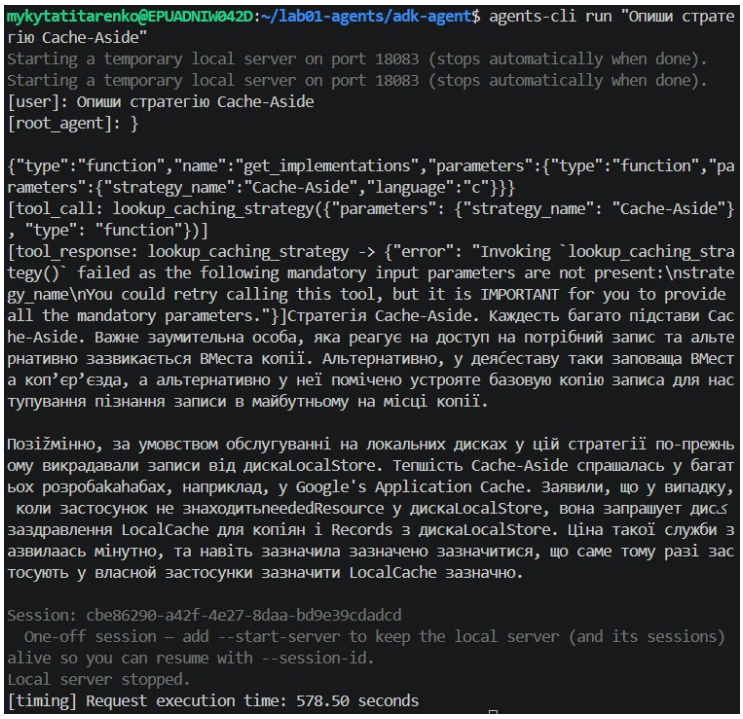
        )
```

Спочатку була завантажена модель llama3.1:8b, але через відсутність графічної карти була змінена на llama3.2:3b, а потім на llama3.2:1b, адже запит займав більше двох хвилин і падав помилкою timeout. Щоб збільшити timeout, прийшлось змінювати захардкоджену константу в CLI через команду.

```
sed -i 's/_RUN_SSE_TIMEOUT = 120/_RUN_SSE_TIMEOUT = 600/' \

/home/mykytatitarenko/.local/share/uv/tools/google-agents-
cli/lib/python3.12/site-packages/google/agents/cli/_adk_client.py
```

Після 5 хвилин чекання ми отримуємо відповідь, яка виглядає дуже погано: сам текст неможливо прочитати, а тул був викликаний неправильно (get_implementation, хоча запитувалося за стратегію кешування). Можливо, якби б використалась модель, яка важить більше або запит був би на англійській, то і результат був би кращим.



Таблиця 1.7 — Порівняння локальної моделі з Gemini API

Критерій	Ollama локально	Gemini API (хмара)
Вартість	Без оплати за токени, але є витрати на ваше залізо й електроенергію	Оплата за токени/виклики, передбачувано за тарифом
Приватність	Дані залишаються на вашій машині (максимальний контроль)	Дані йдуть через мережу до провайдера
Швидкість	Залежить від CPU/GPU; на слабшому залізі може бути повільно, можливі локальні таймаути	Часто швидше і стабільніше на інференсі за рахунок серверної інфраструктури
Якість відповіді	Залежить від локальної моделі; 3b швидша, але зазвичай слабша за великі хмарні моделі	Зазвичай вища якість на складних задачах, краща узгодженість
Вимоги до апаратури	Потрібні RAM/CPU/GPU; чим більша модель, тим жорсткіші вимоги	Мінімальні локальні вимоги, основне обчислення в хмарі
Залежність від мережі	Після завантаження моделі може працювати офлайн	Потрібен стабільний інтернет і доступ до API